

Assignment 7: Decision Tree & Random Forest

Exercise 1: Decision Tree Challenge - Titanic Passenger Survival Analysis.

Exercise 1 - Jupyter notebook code and output.

```
jupyter Exercise-01-Decision-Tree-Challenge.ipynb Last Checkpoint: 3 minutes ago

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[3]: # -----
# Exercise 1 : Decision Tree Challenge
# Titanic Passenger Survival Analysis
# -----

import pandas as pd
import matplotlib.pyplot as plt

# -----
# Step 1 : Create Sample Dataset
# -----

data = {
    'Passenger': ['A', 'B', 'C', 'D'],
    'Sex': ['Female', 'Male', 'Female', 'Male'],
    'Age': [28, 45, 7, 30],
    'Survived': ['Yes', 'No', 'Yes', 'No']
}

df = pd.DataFrame(data)

print("Titanic Sample Dataset")
print(df)

# -----
# Step 2 : Count Survivors by Gender
# -----

survival = df.groupby('Sex')['Survived'].apply(lambda x: (x == 'Yes').sum())

print("\nNumber of Survivors")
print(survival)

# -----
# Step 3 : Plot Bar Graph
# -----

plt.figure(figsize=(8,5))

bars = plt.bar(
    survival.index,
    survival.values,
    edgecolor='black',
    linewidth=1.2
)

plt.title(
    "Decision Tree Challenge: Titanic Passenger Survival by Sex",
    fontsize=14,
    fontweight='bold'
)

plt.xlabel(
    "Passenger Sex",
    fontsize=12
)

plt.ylabel(
    "Number of Survivors",
    fontsize=12
)

plt.grid(axis='y', linestyle='--', alpha=0.5)

# Display values above each bar
for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width()/2,
```

Exercise 1 - Output graph.

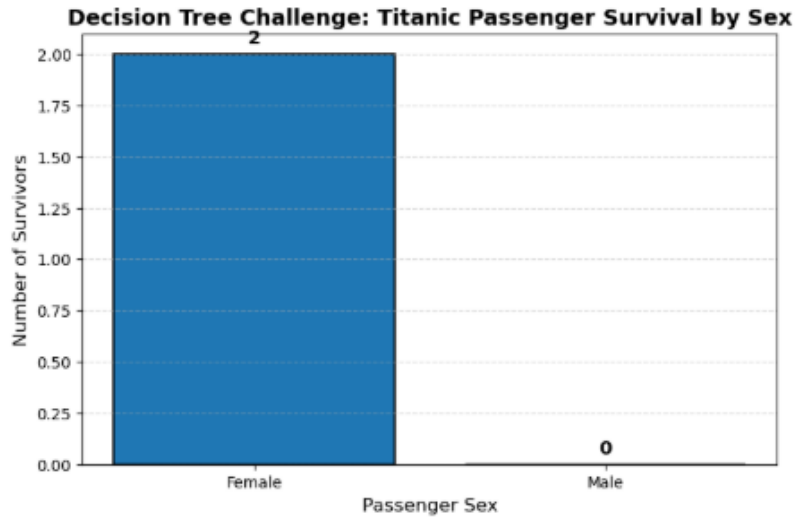
Titanic Sample Dataset

Passenger	Sex	Age	Survived
0	A Female	28	Yes
1	B Male	45	No
2	C Female	7	Yes
3	D Male	30	No

Number of Survivors

Sex	Count
Female	2
Male	0

Name: Survived, dtype: int64



Exercise 2: Random Forest Simulation.

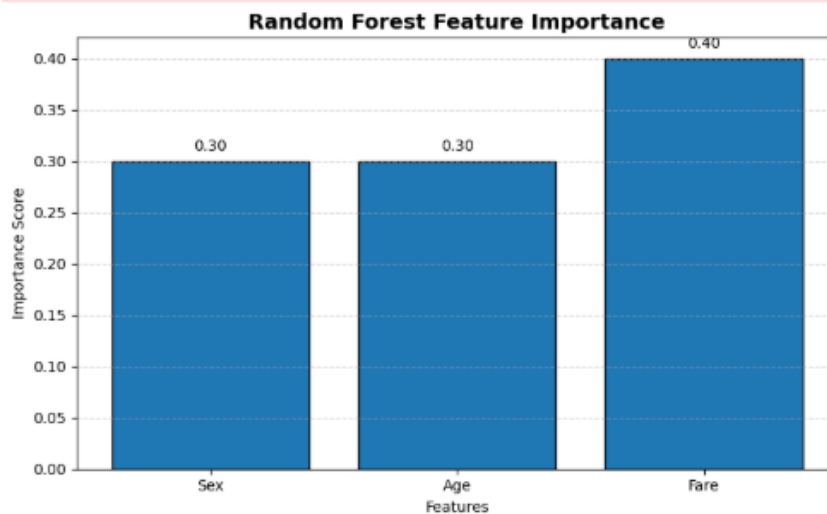
Exercise 2 - Random Forest output and feature importance graph.

Titanic Sample Dataset

Passenger	Sex	Age	Fare	Survived
0	A 1	28	71	1
1	B 0	45	15	0
2	C 1	7	120	1
3	D 0	30	8	0
4	E 1	18	90	1
5	F 0	55	20	0
6	G 1	12	150	1
7	H 0	40	25	0

Prediction : Survived

C:\Users\91953\anaconda3\lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid feature names, but RandomForestClassifier was fitted with feature names
warnings.warn(



Graph saved as random_forest_feature_importance.png

Exercise 2 - Jupyter notebook code.

```
Jupyter Exercise-02-Random-Forest-Simulation.ipynb Last Checkpoint: 1 minute ago
File Edit View Run Kernel Settings Help Trusted
Python 3 (ipykernel)

[1]: # -----
# Exercise 2 : Random Forest Simulation
# Titanic Passenger Survival Prediction
# -----

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.ensemble import RandomForestClassifier

# -----
# Step 1 : Create Sample Dataset
# -----

data = {
    'Passenger': ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H'],
    'Sex': [1, 0, 1, 0, 1, 0, 1, 0], # Female=1, Male=0
    'Age': [28, 45, 7, 30, 18, 55, 12, 40],
    'Fare': [71, 15, 120, 8, 90, 20, 150, 25],
    'Survived': [1, 0, 1, 0, 1, 0, 1, 0]
}

df = pd.DataFrame(data)

print("Titanic Sample Dataset")
print(df)

# -----
# Step 2 : Features and Target
# -----

X = df[['Sex', 'Age', 'Fare']]
y = df['Survived']

# -----
# Step 3 : Train Random Forest
# -----

model = RandomForestClassifier(
    n_estimators=10,
    random_state=42
)

model.fit(X, y)

# -----
# Step 4 : Predict New Passenger
# Female, Age=25, Fare=80
# -----

new_passenger = [[1, 25, 80]]

prediction = model.predict(new_passenger)

if prediction[0]==1:
    print("\nPrediction : Survived")
else:
    print("\nPrediction : Not Survived")

# -----
# Step 5 : Feature Importance
# -----

importance = model.feature_importances_

features = ['Sex', 'Age', 'Fare']

plt.figure(figsize=(8,5))

plt.bar(
    features,
    importance
)
```

Exercise 3: Reflection

Question: If one tree makes a mistake, how does the forest correct it?

Answer: A Random Forest combines the predictions of many decision trees. Each tree is trained on different subsets of data and features. If one tree makes an incorrect prediction, the remaining trees can still vote for the correct class. The final prediction is made using majority voting, which reduces the effect of individual errors, improves accuracy, minimizes overfitting, and makes the model more robust and reliable.